Reproducing the multi-task argument-mining paper: what we found and what we changed

15 October 2026

Paper: Farzam, Shekhar, Mehlhaff & Morucci, *Multi-Task Learning Improves Performance in Deep Argument Mining Models*, ArgMining 2024 (the PDF is at `~/Downloads/Multi-task-learning-og-peper.pdf`).

---

1. The short version

- We had three things that should agree: **the paper's numbers**, **the original research code** (`multitask_basic`, called "legacy" below), and **the `multitask` Python library**. They didn't.
- The paper reports an average score of **66.7**. Both the legacy code and the library came out around **58**. With the paper's exact settings, the library scored only **39**.
- We found **two main reasons**:
  1. **The legacy code never actually uses dropout.** A small bug skips every dropout layer. So the paper's "40% dropout" probably never happened. The library *does* apply 40% dropout, and that badly hurts its small model.
  2. **The library trains the propaganda part of the model differently from the paper.** It gives the model 18 separate, very rare "which propaganda technique is this?" questions. The paper (and the legacy code) gives it **one** question: "is this propaganda at all?".
- Once the library was set up like the paper describes the loss (one pooled propaganda question) and with no dropout (what the legacy code actually did), its first run scored **66.4 vs the paper's 66.7**.
- **Seed check (Section 9):** that first run turned out to be the luckiest of 5. Averaged over 5 random seeds, the score is **64.1 ± 2.2** (range 61.4–66.4). That is about 2.6 points below the paper, compared with about 58 before the change. The ranking measure (AUC) is very stable across seeds: **66.5 ± 0.5**.

---

2. What we compared

Name used here	What it is	Where it lives
Paper	Table 3, "Multi-Task" column	the PDF
Legacy	The authors' original training code	`multitask_basic/` (a copy is in `multitask/replication/legacy_basic/` for running on Modal)
Library	The packaged, cleaned-up version	`multitask/multitask/`, run through `multitask/replication/test_no_prop_overlap.py`

The legacy copy in `replication/legacy_basic/` matches `multitask_basic/` except for:

- one `try/except` around a GPU memory setting, so it can run on Modal;
- the data path in the config.

The training and evaluation code is byte-for-byte the same.

The data: about 219,000 training texts from three sources. Each text is labelled only for its own source.

- **Propaganda**: news sentences, each labelled with 18 propaganda techniques. Paper Table 3 reports one combined "is it propaganda?" score.
- **IAC**: online debate posts, labelled with 8 discourse dimensions, for example disagree/agree and nasty/nice.
- **WA**: argument quality, one label.

That gives 10 reported scores in total.

The score: "weighted F1" (0–100), the same metric as the paper's Table 3. All numbers are on the **test** split. The yes/no cut-off for each task is tuned on the validation split. We also report **ROC AUC**, which measures whether the model ranks positive examples above negative ones, without any cut-off. An AUC of 50 means the model has learned nothing.

> **Why look at AUC as well as F1?** Weighted F1 can look fine even when a model has learned nothing. For example, if 79% of examples are "agree", always guessing "agree" already gets a decent weighted F1. AUC can't be fooled that way.

---

3. First result: the legacy code does not reproduce the paper

With the paper's settings, the legacy code averages **58.2** (a second run gave 57.3), against the paper's **66.7**.

- It comes within about 5 points of the paper on 6 of the 10 tasks.
- On **Propaganda** it scores 28.7 vs 61.7: it labels almost every sentence as propaganda.

While running it, we found three things in the legacy code that differ from what the paper says.

3a. The legacy code never applies dropout (a bug)

Dropout randomly switches off some neurons during training. It's meant to stop the model from memorising the training data. The paper says it uses 40% dropout after every layer.

In `multitask_basic/models/model.py`, each layer is stored as a pair, (dense layer, dropout layer). But the code that runs the model reads:

```
for layer, _ in self.shared_layers:   # the dropout layer goes into "_" and is ignored
    x = layer(x)
```

The `_` means "throw this away", so **the dropout layer is never called**. The same happens in all three parts of the network (lines 73, 85 and 94). The bug is already there in the earliest commit (`f896528`, December 2023), which predates the paper.

Why it matters: if the paper's numbers came from this code, they were produced **without dropout**. The "40% dropout" setting had no effect. That also explains why legacy runs at dropout 0.4 and at 0.1 give the same results (58.2 and 57.9).

3b. The legacy network is much bigger than the paper says

- The paper describes small layers: shared 64→32, then 32→16, then 16→8, about **32,500 extra parameters** on top of BERT.
- The legacy code builds shared 256→256→128→128, then 128→128→64→64, then 64→64→32→32, about **595,000 extra parameters**, roughly 18 times more.
- The library builds the small model the paper describes.

3c. Training stops very early, and warm-up is longer than stated

- **Early stopping.** The paper says training stops "after two epochs without a decrease in loss". The legacy code uses the Keras default of **zero** patience, so it stops the first time validation loss goes up. In our runs it kept the model from **epoch 1 or 2**, out of 10 allowed.
- **Warm-up.** The legacy code computes warm-up steps as `total_steps // 5`, which is **20%** of training, not the 5% in the paper.

---

4. The grid search

The paper says the settings were chosen by "a standard grid search over 72 sets of hyperparameters", picking the one with the best validation F1. The grid isn't listed in the paper, but a commented-out block in the legacy code has it:

Setting	Values tried
learning rate	0.00015, 0.0003
weight decay	0.01, 0.05, 0.1
warm-up	5, 20
dropout	0.1, 0.25, 0.4
batch size	32, 64

2 × 3 × 2 × 3 × 2 = **72**, which matches the paper.

- The grid search was run **once, before this code was written up**. It doesn't run on every training run: the loop is commented out, and the winning values are hardcoded in the config.
- The winning batch size, **256**, isn't in the grid, which only tries 32 and 64. Either the grid was different, or the batch size was changed afterwards. The paper doesn't say.
- Because of the dropout bug, the **dropout part of the grid did nothing**. Picking 0.4 came down to run-to-run randomness.
- The search used one run per setting. Picking the best of 72 runs naturally gives an optimistic number.

So the paper's settings were tuned for the legacy code. That code is a wide network with no dropout, so the same settings shouldn't be expected to suit the library's small network, which does apply dropout.

---

5. Class weights and task weights: how each version balances the loss

The *loss* is the number the model tries to make smaller during training. Each wrong prediction adds to it. **Weights** decide how much each kind of mistake counts.

Class weights (for imbalance within one task)

If only 1 in 100 examples is positive, a lazy model can say "negative" every time and still be 99% right. Class weights make mistakes on the rare class count more, so the model can't ignore it.

	Legacy	Library
Formula	1/frequency, scaled so that w0 + w1 = 1	N / (2 × count), the standard "balanced" formula
Typical size	between 0 and 1	≈ 0.5 for common classes; up to **884** for very rare ones
Applied to	the **10** outputs (propaganda already pooled into one)	all **27** heads, *including the 18 separate propaganda techniques*

Within a single task, both formulas keep the same ratio between the two classes. The difference is size: the library's weights get very large when a class is rare.

Task weights (for balance between data sources)

The paper's formula weights each data source by 1/(its size), written v_k, and splits that weight evenly over the tasks in the source, written 1/|T_k|. This stops the largest source from dominating training.

- **Legacy:** computes these weights (`gamma`) but **never uses them**, because of a second small bug.
- **Library:** applies them exactly as the paper describes.

On this point, the library follows the paper and the legacy code doesn't.

---

6. Dropout experiments in the library

Library setting	Mean F1	Mean AUC
dropout 0.4 (paper setting)	39.2	50.7 (learns nothing)
dropout 0.1	58.7	63.6
dropout 0	57.6	63.5

40% dropout after every layer, including the tiny 16- and 8-neuron layers at the end, is too much for the library's small network, and it stops learning. At 0.1 or 0, it does about as well as the legacy code. But all of these runs were still about **9 points below the paper**, so dropout wasn't the whole story.

---

7. The loss change, and why it helped

7a. What the paper says

The paper describes the output like this (Section 4.1):

> "we apply max-pooling to eighteen entries of the output corresponding to propaganda techniques in order to obtain a binary label. The network outputs a real-valued **10-dimensional** vector which is then mapped to a binary vector of size 10…"

So the model has 18 propaganda-technique heads inside it, but they are merged by taking the **maximum** into **one** propaganda output. The network's output, and therefore what the loss sees, is 10 numbers:

- 1 for propaganda,
- 8 for IAC,
- 1 for WA.

The legacy code does exactly this.

7b. What the library did

The library also computes the merged propaganda output, but **only for evaluation**. During training, its loss looks at the 18 technique heads **one by one**, each against its own label. So the library's model is effectively asked 18 hard questions ("is this Bandwagon?", "is this a Straw Man?", …) rather than the one question that gets scored ("is this propaganda?").

7c. Why 18 separate questions was a problem

Most propaganda techniques are **extremely rare**. A training batch has 256 texts, and about 72 of them are propaganda-corpus sentences. Here is what a typical batch contains:

Label	% positive	Positives per batch (average)	Batches with no positive at all	Library class weight for a positive
Bandwagon	0.06%	0.04	**96%**	**884**
Straw Men	0.08%	0.06	95%	645
Red Herring	0.17%	0.12	89%	298
Loaded Language	13.2%	9.5	0%	3.8
Pooled "any propaganda"	**36.6%**	**26.5**	**0%**	**1.37**

10 of the 18 techniques are positive less than 1% of the time.

Here is what that does to training, step by step:

1. **Most batches teach the rare heads nothing useful.** For Bandwagon, 96% of batches contain no example of it, so all the head learns is "say no".
2. **Then, now and then, a huge jolt.** When a Bandwagon example does appear, its weight of 884 makes it count as much as about 1,700 ordinary examples put together. That one example suddenly dominates the whole update.
3. **The jolt spreads through the whole network.** The 18 heads sit on top of layers shared with every other task. These rare, oversized updates hit the shared layers too, which makes training noisy for *all* tasks, not just propaganda.
4. **The tasks being learned aren't the one being scored.** Even if the 18 heads learn something, the test score asks "is it propaganda?", and the model was never trained directly on that question.

An analogy: imagine teaching a student to spot fraud by drilling them on 18 rare, specific fraud types. Most days none of them come up. Then, once in a while, one does, and you shout about it at full volume. The exam only asks "is this fraud, yes or no?", which you never actually taught.

7d. What we changed

We changed **only the propaganda part of the loss**, keeping everything else exactly as the library does it:

- **Before:** 18 separate loss terms, one per technique, each against its own label, with weights up to 884.
- **After: one** loss term. The prediction is the maximum of the 18 heads' outputs, which is the same number the model reports as `propaganda_max_pool`. The label is "1 if any technique is present". The class weights are calculated for that merged label: 0.79 for "no" and 1.37 for "yes".
- The 8 IAC tasks and WA keep exactly the same loss terms and weights as before.
- Propaganda keeps its paper-style source weight v_k, now counted as **one** task (|T_k| = 1) instead of 18.

7e. Why that helps

1. **Steady signal instead of rare jolts.** Every batch now has about 26 positive and 46 negative propaganda examples, with modest weights. The model gets a steady, sensible update for propaganda on every step.
2. **It trains on the question that's scored.** The loss now targets exactly the output that is tested.
3. **The shared layers settle down.** Without the oversized updates, the layers shared by every task learn a more useful representation. That's why tasks whose loss we **didn't change** also improved, such as Attacking/Respectful, Personal/Audience, Defeater/Undercutter and Disagree/Agree.
4. **The 18 heads are still trained, just indirectly.** Because of the max, on each propaganda example only the highest-scoring head gets the update (we tested this; see Section 8). We haven't checked whether the heads end up matching the individual techniques. That doesn't affect the scored output, but it means the per-technique outputs shouldn't be read as technique predictions.

7f. The results

Weighted F1 on the test set (ROC AUC in brackets), all with seed 42:

Task	Paper	Library, pooled loss, dropout 0	Library, pooled loss, dropout 0.4	Library, original loss, dropout 0	Library, original loss, dropout 0.4	Legacy
Propaganda	61.7	**65.7** (68.9)	61.0 (59.0)	48.3 (52.4)	42.8 (49.3)	28.7
Disagree/Agree	71.4	64.2 (55.5)	54.1 (51.4)	37.7 (54.0)	68.0 (50.0)	66.7
Emotion/Fact	63.9	66.3 (73.8)	61.1 (64.4)	66.3 (72.4)	57.8 (55.1)	61.1
Attacking/ Respectful	68.1	63.1 (67.2)	47.8 (56.5)	58.2 (66.0)	31.8 (51.3)	56.5
Nasty/Nice	73.7	68.9 (70.1)	50.2 (59.9)	65.9 (68.3)	34.5 (52.9)	52.1
Personal/ Audience	65.7	67.9 (59.5)	63.9 (56.5)	51.7 (58.8)	66.5 (50.2)	61.0
Defeater/ Undercutter	55.7	59.3 (59.6)	43.7 (54.7)	47.9 (57.0)	24.8 (48.5)	46.2
Negotiate/Attack	64.8	66.8 (72.1)	61.7 (64.5)	65.8 (70.1)	39.4 (51.2)	62.0
Questioning/ Asserting	63.2	58.2 (58.4)	50.8 (59.7)	56.2 (57.9)	21.8 (48.0)	66.1
Argument Quality (WA)	79.2	83.4 (77.9)	73.1 (72.5)	78.2 (77.9)	4.2 (50.6)	82.2
Average	**66.7**	**66.4 (66.3)**	56.7 (59.9)	57.6 (63.5)	39.2 (50.7)	58.2
Average gap to paper		**4.0**	10.0	9.8	27.7	9.7

How to read this:

- **Pooled loss with no dropout comes closest to the paper** in this run, both on average (66.4 vs 66.7) and task by task (average gap 4 points, versus about 10 for every other setup). But seed 42 was the best of 5 seeds; the 5-seed average is 64.1 (Section 9).
- **Propaganda goes from guessing to learning.** AUC rises from 52 to 69.
- **Dropout 0.4 still hurts**, even with the pooled loss: about 10 points lower on average. The paper's "40% dropout" doesn't suit this model. That fits Section 3a, which suggests the paper's results were produced without dropout.

---

8. Every code change we made

The `multitask` library itself (`multitask/multitask/`) was **not changed**. Everything is opt-in, in replication files that git doesn't track. Backups of the originals are in the scratchpad.

File	Change	Why
`replication/test_no_prop_overlap.py`	Added `compute_full_train_pooled_class_weights()`	Calculates class weights for the merged "any propaganda" label from the training CSV. It treats both `NA` and `-1` as missing. The first version missed `-1` and crashed before training; that was fixed before any results were produced.
same	Added `PooledOutputLossTrainer`, a subclass of the library's `Trainer` that replaces only its loss-building step	Puts the propaganda loss on the max-pooled output instead of the 18 heads, while keeping every other task's loss identical to the library's
same	`run_experiment(..., trainer_factory=None)`	Lets an experiment supply its own trainer. When it's not given, nothing changes.
same	New flag `--pooled-propaganda-loss`	Turns the new loss on. Off by default.
`modal_replicate.py`	New option `--pooled-loss`	Passes that flag through when running on Modal
`replication/test_pooled_output_loss.py`	**New test file**, 3 tests, all passing on Modal	(1) In the simplest case, a "pool" of one head, the new loss gives **exactly** the library's original loss, which proves the weighting code matches. (2) The new loss matches a hand calculation of the paper's formula. (3) Gradients reach only the highest-scoring propaganda head in each row, and nothing for rows without propaganda labels.
`replication/legacy_basic/configs/config_iac_modal_d04.yaml`, `..._d01.yaml`	Copies of the legacy config with dropout 0.4 or 0.1 and a distinct output name	To run the legacy code at both dropout settings

How to run the main result:

```
cd multitask
modal run modal_replicate.py::replicate --data-dir /data/paper \
    --dropout 0.0 --seed 42 --pooled-loss --tag pooledloss_d00_seed42
```

Results are written to the `multitask-outputs` Modal volume as `paper_replication_<tag>.json`.

---

9. Seed check (how much is luck?)

Neural-network training involves randomness: the starting weights, the order of the data, and which neurons dropout switches off. Running the same setup with different random **seeds** shows how much of a score is the setup and how much is luck.

We ran the pooled-loss setup with **5 seeds at dropout 0** (42, 1, 2, 3, 4) and **3 seeds at dropout 0.1** (42, 1, 2).

Average over all 10 tasks:

Setting	Seeds	Mean F1 (± sd)	Range of F1	Mean AUC (± sd)
Paper	grid search, best of 72	66.7	—	—
Pooled loss, dropout 0	5	**64.1 ± 2.2**	61.4 – 66.4	**66.5 ± 0.5**
Pooled loss, dropout 0.1	3	**64.2 ± 0.7**	63.4 – 64.7	**65.7 ± 0.7**
For reference (1 seed each): original loss, dropout 0 / 0.1 / 0.4	1	57.6 / 58.7 / 39.2	—	63.5 / 63.6 / 50.7
For reference (1 seed each): legacy	1	58.2 / 57.3	—	—

The per-seed mean F1 at dropout 0 was 66.4, 66.2, 62.5, 64.1 and 61.4. The per-seed mean AUC was 66.3, 67.1, 65.9, 66.8 and 66.2.

Per task, pooled loss, dropout 0, 5 seeds:

Task	Paper	F1 mean ± sd	F1 range	AUC mean ± sd
Propaganda	61.7	**64.8 ± 0.7**	64.0 – 65.7	68.3 ± 0.7
Disagree/Agree	71.4	47.9 ± **17.5**	22.5 – 64.3	56.0 ± 2.0
Emotion/Fact	63.9	67.7 ± 1.2	66.3 – 69.3	74.6 ± 0.6
Attacking/Respectful	68.1	63.7 ± 2.2	61.3 – 66.5	66.8 ± 0.5
Nasty/Nice	73.7	68.0 ± 2.7	63.8 – 71.0	70.3 ± 0.5
Personal/Audience	65.7	66.0 ± 2.5	62.3 – 68.1	59.9 ± 0.5
Defeater/Undercutter	55.7	55.9 ± 3.0	52.7 – 59.3	57.0 ± 2.0
Negotiate/Attack	64.8	67.0 ± 1.2	65.1 – 68.0	72.5 ± 0.6
Questioning/Asserting	63.2	60.9 ± 3.2	56.8 – 64.4	60.6 ± 1.3
Argument Quality (WA)	79.2	79.0 ± 3.5	75.3 – 83.4	78.8 ± 0.9

What this tells us:

1. **Our first result (66.4) was on the lucky side.** Seed 42 happened to be the best of the five. A fair summary is **about 64**: roughly 2.6 points below the paper, but about 6 points above everything we had before the loss change.

2. **The propaganda improvement is real.** Propaganda F1 is 64.8 ± 0.7 and AUC 68.3 ± 0.7 on every seed, against AUC about 50–52 with the original loss. This is the clearest effect of the change.
3. **The model learns almost the same thing every time.** The average AUC varies by only ±0.5 between seeds. Most of the variation in F1 comes from **where the yes/no cut-off lands**, not from what the model learned.
4. **One task causes most of the spread.** Disagree/Agree F1 ranges from 22.5 to 64.3, while its AUC barely moves (56 ± 2). About 79% of its examples are "agree", and its AUC is low, so the tuned cut-off can flip the model between saying "agree" to almost everything and saying "disagree" to most things. This one task moves the 10-task average by up to 4 points.
5. **Dropout 0.1 is as good as dropout 0** (64.2 vs 64.1), and steadier across seeds. Only the heavy 0.4 hurts.
6. **The paper's 66.7 is the best of a 72-run grid search**, so it should be compared with our *best* runs (66.4 and 66.2), not our average. Seen that way, the library with the pooled loss reproduces the paper's level.

9a. The paper's dropout (0.4) against the paper

The paper says it used 40% dropout, so we also checked how the pooled loss does at 0.4 (one seed, 42). The paper only reports weighted F1, so its AUC cells are empty.

Setup	Mean F1	Mean AUC	Propaganda F1 / AUC	WA F1	Gap to paper (mean F1)
Paper (Table 3, best of 72-run grid search)	**66.7**	—	61.7 / —	79.2	—
Library, original loss, dropout 0.4 (seed 42)	39.2	50.7	42.8 / 49.3	4.2	–27.5
Library, pooled loss, dropout 0.4 (seed 42)	**56.7**	**59.9**	61.0 / 59.0	73.1	**–10.0**
Library, pooled loss, dropout 0 (seed 42)	66.4	66.3	65.7 / 68.9	83.4	–0.3
Library, pooled loss, dropout 0 (mean of 5 seeds)	64.1 ± 2.2	66.5 ± 0.5	64.8 / 68.3	79.0	–2.6

What this tells us:

- **The pooled loss rescues dropout 0.4.** With the original loss, the model at 0.4 was guessing on every task (AUC about 50, WA F1 4.2). With the pooled loss, all ten tasks learn something, and Propaganda F1 (61.0) is within 0.7 of the paper.
- **But dropout 0.4 still costs about 10 points**, both against the paper and against the same seed at dropout 0. Every task is lower, and training is slow (the best epoch was 8, against 2–3 at lower dropout). The paper-sized network is tiny (64 → 32 → 16/8 units), and switching off 40% of such small layers after every dense layer removes too much signal.
- **We only come close to the paper at dropout 0 or 0.1.** This fits section 3a: the legacy code never actually applied dropout, so the paper's numbers were most likely produced without it.
- The 10-point gap is much bigger than the ±2.2 seed-to-seed spread, so one seed is enough for this conclusion.

Caveat: the reference rows (original loss, legacy) are single runs. We didn't re-run them with several seeds, so their spread is unknown. The propaganda and AUC gains are far larger than the seed-to-seed variation we measured, so we're confident about those. The size of the overall F1 gain (about 6 points) is less certain.

---

10. What is still uncertain

- **One version of the truth.** We assume the paper's numbers came from the legacy code as it is in git. The authors might have run a slightly different version on their cluster. Given the evidence (same repository, bug present since the first commit, legacy results unchanged by dropout), "the paper's results were produced without dropout" is the most likely explanation, but it isn't proven.
- **Per-task numbers are noisy.** A single task can move 10–40 F1 points between runs (Disagree/Agree ranged from 22.5 to 64.3 across 5 seeds), because the yes/no cut-off is re-tuned each time. Averages, and AUC, are much more reliable than individual task scores.
- **A small remaining gap.** Averaged over seeds, the pooled-loss library is about 2.6 points below the paper. The paper's number is the best of a 72-run grid search, and our best seeds come within 0.5 points of it, so this gap is consistent with the paper reporting a lucky or tuned run.
- **Other legacy differences we haven't tested separately:** the much bigger network, unused task weights, early stopping at patience 0, and 20% warm-up. Our library result comes close to the paper without copying any of these, which suggests they aren't essential.

11. Suggested next steps

1. Fix the dropout bug in the legacy code, by applying each dropout layer after its dense layer, and rerun at dropout 0.4. This would show directly what the paper's described model really scores.
2. Consider making the pooled propaganda loss an option in the library itself. It matches the paper's description and clearly trains better.
3. When reporting, use a few seeds and show the spread, and report AUC as well as weighted F1.
4. Run the original-loss setup with the same 5 seeds, to measure the size of the loss change's gain with error bars on both sides.

Appendix: where every result lives

All results are on the Modal volume `multitask-outputs`.

Run	File
Library, paper settings (dropout 0.4)	`paper_replication_paper_seed42.json`
Library, dropout 0.1	`paper_replication_dropout0.1_seed42.json`
Library, dropout 0	`paper_replication_dropout0_seed42.json`
Library, pooled loss, dropout 0	`paper_replication_pooledloss_d00_seed42.json` (+ seeds 1–4)
Library, pooled loss, dropout 0.4	`paper_replication_pooledloss_d04_seed42.json`
Library, pooled loss, dropout 0.1	`paper_replication_pooledloss_d01_seed{42,1,2}.json`
Legacy, dropout 0.4 (13 Oct)	`legacy_basic/metrics_argument_2026_10_14_03_24_27.json`
Legacy, dropout 0.4 (rerun)	`legacy_basic/metrics_argument_d04_*.json`
Legacy, dropout 0.1	`legacy_basic/metrics_argument_d01_*.json`